CSCI 532 - Algorithms Homework 3

Ben Heinze

April 28, 2026

1 Problem 1.

The maximum-weight matching problem on a bipartite graph is as follows: Given a bipartite graph with edge weights, select the subset of edges with the largest total weight such that no edges share a node. Formulate this problem as an ILP.

2 Solution 1

Decision Variables: Which edge we choose to select:

$$x_{ij} \in \{0, 1\} = \text{ indicates if edge } ij \text{ is selected.}$$

Objective: We want to maximize sum of the edge weights, where edge $w(i, j)$ is the weight of edge e between nodes i and j.

$$max \sum x_{ij} \cdot w_{ij}$$

Constraints: The nodes connecting edge x_{ij} can be selected at most, once. To do this, we will constrain the sum of all of node x_i's edges to at most 1, then do the same thing for node x_j. Since the graph is bipartite, we will define this for both the top layer and the bottom layer:

$$\sum_{j \in neighbor(i)} x_{ij} \leq 1, \text{for all } i \text{ in top layer}$$

$$\sum_{i \in neighbor(j)} x_{ij} \leq 1, \text{for all } j \text{ in bottom layer}$$

3 Problem 2.

If you remove the integer restriction on variables in the ILP from the previous problem, will the resulting LP still find valid optimal solutions to the maximum-weight matching problem on bipartite graphs? Why or why not?

4 Solution 2.

If we relax our ILP to an LP, our decision variable's integer restraint is relaxed to contain all real numbers:

$$x_{ij} \in [0,1] = \text{ indicates if edge } ij \text{ is selected.}$$

Now that we've relaxed edges so they can have a value between $0 \leq x_{ij} \leq 1$

We can create a matrix A where the columns are edges x_{ij} and the rows are nodes.

$A_{ij} = 1$ if the node in that given row is used for edge x_{ij} The right hand of our constraints will always be integer, which is good for the subject $Ax \leq b$, so b is always an integer.

We learned in class that the Ghouila-Houri property is linked to the Hoffman-Kruskal theorem for total unimodularity. If a matrix has the Ghouila-Houri property, then it has an integer solution.

Ghouila-Houri property: $A \in \mathbb{R}^{m \times n}$ is totally unimodular $\iff$ for every subset of rows R, there is a partition $R = R_1 \cup R_2$ such that for every column $j, \sum_{r \in R_1} A_{ij} - \sum_{r \in R_2} A_{ij} \in \{-1, 0, 1\}$

To address the Ghouila-Houri property in our constructed A matrix, we need to show that for each column, which represents an edge x_{ij}, every possible partition of rows $R = R_1 \cup R_2$, subtracting the total counts from each subset results in a -1, 0, or 1. To guarantee that each node is put in a different subset, we construct the partition by using 2-color such that R is partitioned into $R_1 = R \cap B$ and $R_2 = R \cap G$. By alternating the generations of vertices into a blue set B and a green set G, each column will have exactly two 1's, where one node is from R_1 (B) and the other is from R_2 (G). This is guaranteed to work since the graph is bipartite.

- Case 1: $R_1 = 0, R_2 = 0 \rightarrow 0 - 0 = 0$
- Case 2: $R_1 = 0, R_2 = 1 \rightarrow 0 - 1 = -1$
- Case 3: $R_1 = 1, R_2 = 0 \rightarrow 1 - 0 = 1$
- Case 4: $R_1 = 1, R_2 = 1 \rightarrow 1 - 1 = 0$

Every case is proves each result is either a -1, 0, or 1, this shows our matrix A has the Ghoula-Houri property, and therefore has total unimodularity. Since A is totally unimodular and the right hand side of our constraints (vector b) is integer valued, the optimal solution is also integer.

5 Problem 3.

Suppose we have n tasks we want completed and w identical workers who can complete them. Each task has a time t_i it takes a worker to complete. Let A_w denote the tasks assigned to worker w. This means that worker w's work time will be $T_w = \sum_{i \in A_w} t_i$. The goal is to assign all tasks to workers in such a way as to minimize the work time of the worker with the largest work time. I.e., minimize $\max_w T_w$.

Consider the following algorithm:

For each task (in arbitrary order), assign the task to the worker who currently has the smallest work time (smallest current T_w).

Show that this algorithm is a 2-approximation algorithm.

Hint: First show that $\frac{1}{w}\sum_i t_i \leq OPT$, then consider the last task assigned to the worker responsible for the algorithm's cost. Why was that task assigned to that worker?

6 Solution 3.

Our algorithm distributes tasks to each worker. No matter how tasks are assigned, the busiest worker must do at least the average amount of work:

$$\frac{1}{w}\sum_i t_i \leq OPT$$

Now that we've shown this inequality for the average worker, we need to consider the last task assigned to the worker responsible for the algorithm's cost. This task was assigned to the worker because he had the smallest work time.

The busiest worker in the optimal solution is required to work at least t_i because every task must be assigned to someone, whoever receives task i works at least t_i

$$t_i \leq OPT$$

Before the smallest-timed worker $T_{wsmallest}$ was assigned the last task $\$t_{\text{last}}$, his load was at most average:

$$T_{wsmallest} - t_{last} \leq \frac{1}{w}\sum_i t_i \leq OPT$$

We now have upper bounds on two pieces: $(T_{w_{smallest}} - t_{last})$ and t_{last}.

Since $T_{w_{smallest}}$ is just those two pieces added together, we can bound the total:

$$T_{w_{smallest}} = (T_{w_{smallest}} - t_{last}) + t_{last} \leq OPT + OPT = 2 \cdot OPT$$

Therefore the algorithm is a 2-approximation.

7 Problem 4.

Consider the following algorithm for the Knapsack problem discussed in class: For each item, calculate its value to weight ratio and greedily fill the knapsack with the highest valued item at each iteration. What is the approximation ratio for this algorithm? Explain why that is the case.

8 Solution 4.

To find the best constant for α, we can create two items for the knapsack problem, where the knapsack has a weight capacity of 1:

- Item A: weight $= \epsilon$, value $= 2\epsilon$, where $0 < \epsilon < 1$.
- Item B: weight $= 1$, value $= 1$.

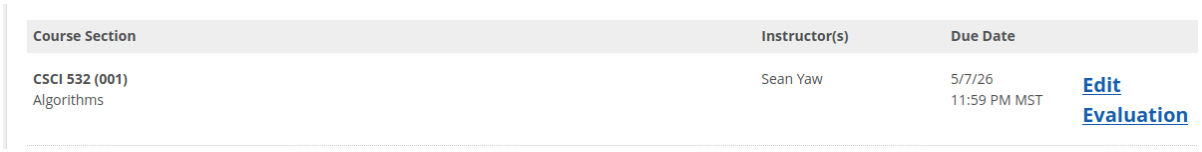
The greedy ALG will always choose item A first because we constructed item A to always have a higher value:weight ratio, and item B will always be optimal. The greedy algorithm can never fit both items because the knapsack's weight capacity is 1 and item B's weight is 1.

As we shrink ϵ's value towards 0, the approximation ratio $\frac{ALG}{OPT}$ gets arbitrarily bad:

$$\frac{v_{ALG}}{v_{OPT}} = \frac{2\epsilon}{1}, \epsilon \text{ approaches zero.}$$

No matter what constant we assign α, there will always be an ϵ that breaks it, meaning the approximation ratio is unbounded.

9 Solution 9.



Course Section	Instructor(s)	Due Date	
CSCI 532 (001) Algorithms	Sean Yaw	5/7/26 11:59 PM MST	**Edit Evaluation**